

Document number: P0163R0

Date: 2015-10-23

Project: ISO JTC1/SC22/WG21, Programming Language C++

Audience: Library Evolution Working Group

Reply to: Arthur O'Dwyer <arthur.j.odwyer@gmail.com>

shared_ptr::weak_type

[1. Introduction](#)

[2. Problem: Generic programming and weak_ptr](#)

[3. One more motivating example](#)

[4. Solution](#)

[5. Alternatives rejected at Kona](#)

[6. Proposed wording](#)

[6a. Proposed wording for C++17 \[util.smartptr.shared\]](#)

[6b. Proposed wording for Library Fundamentals V2 \[memory.smartptr.shared\]](#)

[7. References](#)

[8. Thanks](#)

1. Introduction

This paper identifies a minor inconvenience in the design of `shared_ptr`, in that it is impossible to create a `weak_ptr` from a `shared_ptr` without explicitly spelling out the type of the `weak_ptr`, which impedes generic programming. N4537 [\[N4537\]](#) proposed to solve this problem by introducing the member function `shared_ptr::unlock()`, but this idea was rejected by LEWG at Kona. LEWG strongly supported the idea of providing a member typedef `shared_ptr::weak_type`, so that's what the present paper proposes, for both C++17 and Library Fundamentals V2.

2. Problem: Generic programming and weak_ptr

Given an arbitrary `shared_ptr`, write code to "weaken" the object into a `weak_ptr` with the same shared ownership and stored pointer.

```
template<class ObjectType>
void register_observers(ObjectType& obj)
{
    auto sptr = obj.get_shared_ptr(); // for example, via shared_from_this
    auto wptr = weak_ptr<ObjectType>(sptr);
    sptr.reset(); // drop the strong reference as soon as possible
    register_observer_1(wptr);
    register_observer_2(wptr);
}
```

Unfortunately, this code is not perfectly generic: it will fail (or at least do the wrong thing) in the case that `obj.get_shared_ptr()` returns something other than `shared_ptr<ObjectType>`. For example, it might return `shared_ptr<BaseClassOfObjectType>`. Or, in an advanced scenario, we might want to be able to "drop in" a replacement class `custom_shared_ptr` instead of the standard `shared_ptr`, in which case we would also have to replace `weak_ptr` in the above code with `custom_weak_ptr`.

Notice that at least two values of "custom_shared_ptr" exist in the wild: `std::experimental::shared_ptr` in Library Fundamentals [\[N4077\]](#), and `boost::shared_ptr`. Neither of these implementations currently provide `weak_type`. This paper proposes adding `weak_type` to both C++17 and Library Fundamentals V2.

Notice that the *only* place in the code sample where an explicit type is used, is in the line concerned with "weakening" `sptr`. "Weakening" a `shared_ptr` into a `weak_ptr` is not an operation that ought to force explicit types into otherwise generic code.

3. One more motivating example

A real-world example of this pattern cropped up recently when Arthur attempted to implement a "task-based programming" library with task cancellation, as described in Sean Parent's "Better Code: Concurrency" [\[Parent\]](#). In this library, the `TaskControlBlock` is the central concept; a `Future` is simply a thin wrapper around a `std::shared_ptr<TaskControlBlock>`, and a `CancellablePackagedTask` is a thin wrapper around a `std::weak_ptr<TaskControlBlock>`. When the last `Future` referring to a `TaskControlBlock` is destroyed, the `TaskControlBlock` itself (if the task has not yet begun to execute) may be destroyed. The implementation of `EnqueueTask` in this system looks like this:

```

template<typename T>
inline std::weak_ptr<T> Unlock(const std::shared_ptr<T>& sptr)
{
    return std::weak_ptr<T>(sptr);
}

template<typename Func>
auto EnqueueTask(Func&& func) -> Future<decltype(func())>
{
    using T = decltype(func());

    auto sptr = std::make_shared<TaskControlBlock<T>>();

    auto task = [
        func = std::forward<Func>(func),
        wptr = Unlock(sptr)
   ]() {
        Promise<T> px { wptr };
        px.set_value(func());
    };

    GlobalThreadPool.submit(task);

    Future<T> fx { std::move(sptr) };
    return fx;
}

```

Notice the use of the hand-coded free function `Unlock(sptr)`; that's a workaround for the unwieldy expression `std::weak_ptr<TaskControlBlock<T>>(sptr)`.

4. Solution

We propose adding one new typedef to the standard library. `shared_ptr` gains a member typedef

```
using weak_type = weak_ptr;
```

This allows us to rewrite the "problem" code above in generic style, naming no types explicitly, as:

```

template<class ObjectType>
void register_observers(ObjectType& obj) {
    auto sptr = obj.get_shared_ptr(); // for example, via shared_from_this
    auto wptr = typename decltype(sptr)::weak_type{sptr};
    sptr.reset(); // drop the strong reference as soon as possible
    register_observer_1(wptr);
    register_observer_2(wptr);
}

```

and the other example becomes:

```

auto task = [
    func = std::forward<Func>(func),
    wptr = typename decltype(sptr)::weak_type{sptr}
]() {
    Promise<T> px { wptr };
    px.set_value(func());
};

```

The author is not happy with the proposed style's verbosity, but at least this does solve the generic-programming problem. Also, this proposal avoids introducing any further asymmetry (as would be the case if we introduced a free function `std::weaken` without `std::strengthen`).

5. Alternatives rejected at Kona

At Kona (October 2015), the following was proposed and/or straw-pollled as N4537: [\[Issues\]](#)

Do we want `sptr.unlock()` by that name, as proposed in N4537?
(consensus was against)

Do we want that functionality at all, e.g. under the name `std::weaken(sptr)`?
SF F N A SA
1 1 6 6 1

Should we provide `shared_ptr<T>::weak_type`?
SF F N A SA
0 10 5 0 1

Therefore, this paper proposes only the member typedef that garnered support.

6. Proposed wording

6a. Proposed wording for C++17

The wording in this section is relative to WG21 draft N4527 [\[N4527\]](#), that is, the draft of the C++17 standard.

20.8.2.2 Class template `shared_ptr` [`util.smartptr.shared`]

Edit paragraph 1 as follows.

```
template class shared_ptr {
public:
    typedef T element_type;
    typedef weak_ptr<T> weak_type;

    // 20.8.2.2.1, constructors:
```

6b. Proposed wording for Library Fundamentals V2

The wording in this section is relative to WG21 draft N4529 [\[N4529\]](#), that is, the draft of Library Fundamentals V2.

8.2.1 Class template `shared_ptr` [`memory.smartptr.shared`]

Edit paragraph 1 as follows.

```
template class shared_ptr {
public:
    typedef typename remove_extent_t<T> element_type;
    typedef weak_ptr<T> weak_type;
    // 8.2.1.1, shared_ptr constructors
```

7. References

[Issues]

"Bug 103: Adding Symmetry Between `shared_ptr` and `weak_ptr`" (October 2015).

https://issues.isocpp.org/show_bug.cgi?id=103.

[N4077]

Jonathan Wakely. "Experimental `shared_ptr` for Library Fundamentals TS" (June 2014).

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4077.html>.

[N4527]

"Working Draft, Standard for Programming Language C++" (May 2015).

<http://isocpp.org/files/papers/n4527.pdf>.

[N4529]

"Working Draft, C++ Extensions for Library Fundamentals, Version 2" (May 2015).

<http://isocpp.org/files/papers/n4529.pdf>.

[N4537]

Arthur O'Dwyer. "Adding Symmetry Between `shared_ptr` and `weak_ptr`" (May 2015).

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2015/n4537.html>.

[Parent]

"Better Code: Concurrency" (February 2015).

<https://github.com/sean-parent/sean-parent.github.io/wiki/Papers-and-Presentations>.

8. Thanks

Thanks to LEWG for feedback on N4537, and to Jonathan Wakely for feedback and for the correct spelling of his surname.